Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Nombre del Estudiante	
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet
Guía de práctica.
Plantilla de matriz comparativa

5. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- **Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).**

  Definición: Patrón donde publicadores envían mensajes a un broker sin conocer a los suscriptores; el broker los enruta por topics/colas a los consumidores interesados. Desacopla productores y consumidores en tiempo, espacio y sincronía.

  Componentes:

  - Publicadores (vehículos/sensores)
  - Broker MQTT (Mosquitto): recibe telemetría por topics flota/#
  - Broker AMQP (RabbitMQ): exchanges, bindings, colas duraderas
  - Suscriptores (suscriptor_sqlite.py, mqtt_rabbitmq_bridge.py)
  - Colas: cola.gps.telemetria, cola.alertas.temperatura, cola.combustible.nivel
  - Almacenamiento: SQLite (telemetria.db)

  Ventajas: desacoplamiento, escalabilidad horizontal, tiempo real, persistencia de colas, filtrado por topics.

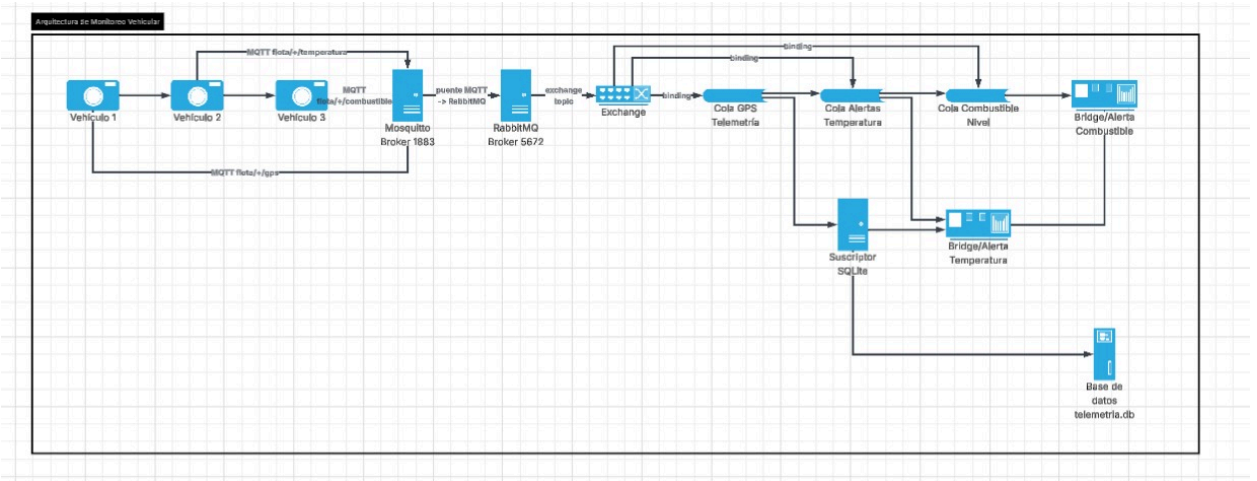
  Desventajas: punto único de fallo del broker (si no hay cluster), complejidad operativa, latencia extra, debugging distribuido más difícil.

  Escalabilidad: horizontal mediante clustering de brokers, particionado de topics, colas por consumidor.

  Ejemplos:

  1. Telemetría de flotas de vehículos (como esta práctica)
  2. Notificaciones push / chat en tiempo real (WhatsApp, Firebase Cloud Messaging)

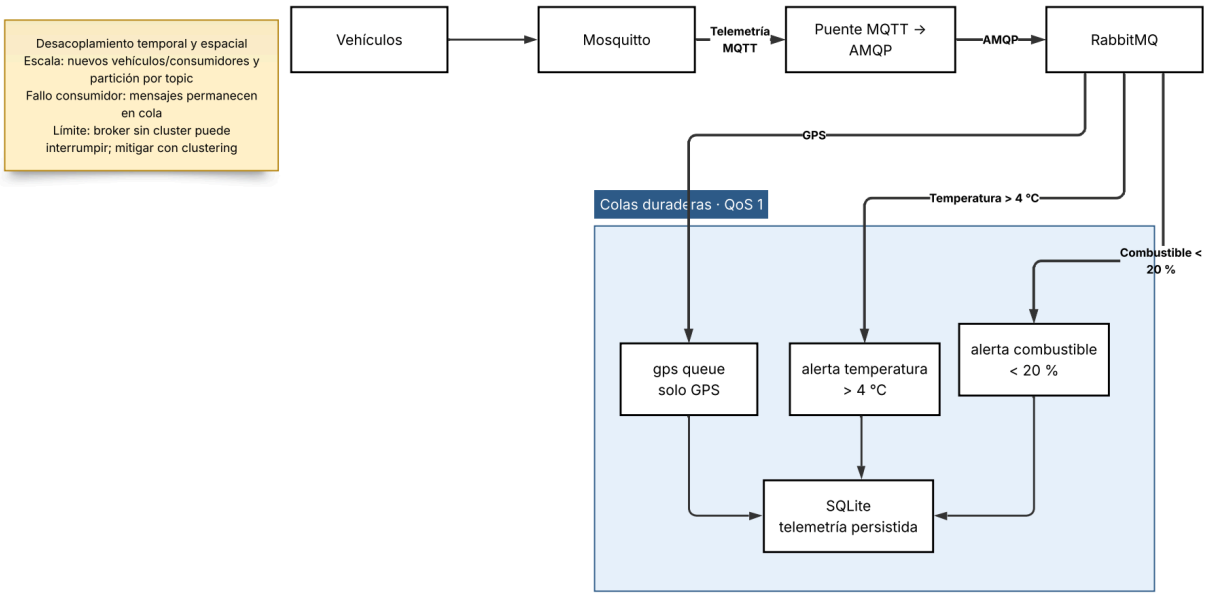
- Un diagrama digital de la arquitectura asignada (draw.io o similar).



- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).

Criterio	Cliente-Servidor	Peer-to-Peer	Microservicios	Pub/Sub con Broker
Definición	Clientes piden, servidores responden	Nodos igualitarios, comparten recursos	Servicios independientes desplegables	Publicadores /suscriptore s vía broke
Acoplamiento	Alto (cliente conoce servidor)	Bajo/Medio	Muy bajo (APIs)	Muy bajo (no se conocen)
Escalabilidad	Vertical del servidor	Horizontal difícil	Horizontal por servicio	Horizontal (colas/topics)
Tolerancia a fallos	SPOF en servidor	Alta (nodo cae, otros siguen)	Alta (aislamiento de fallos)	Media-Alta (cola aguanta, broker SPOF si no cluster)
Transparencia	De acceso/ubicación	De ubicación	De ubicación	De tiempo y espacio
Ejemplo real	Web tradicional	BitTorrent	Netflix, Amazon	Telemetría IoT, notificacione s push
Complejidad	Baja	Media	Alta	Media

- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.



- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

  Reporte Individual: Tayron Morales

  Introducción:

  En esta práctica se estudiaron cuatro arquitecturas distribuidas: cliente-servidor, peer-to-peer, microservicios y arquitectura orientada a mensajes con brokers (pub/sub), la última implementada en la práctica APE5 con MQTT, RabbitMQ y SQLite para telemetría de vehículos.

  Desarrollo:

  La arquitectura cliente-servidor se basa en peticiones síncronas de clientes a un servidor central. Es sencilla pero acopla fuertemente a los componentes y depende de la disponibilidad del servidor.

  El modelo peer-to-peer distribuye responsabilidades entre nodos igualitarios, lo que evita puntos únicos de falla, pero dificulta la coordinación, auditoría y gestión centralizada de telemetría.

  Los microservicios dividen la aplicación en servicios independientes desplegables por separado. Ofrecen escalabilidad y aislamiento de fallos, pero añaden complejidad operativa y de comunicación entre servicios.

  La arquitectura pub/sub con brokers desacopla publicadores y suscriptores en tiempo y espacio. En la práctica APE 5, los vehículos publican telemetría en Mosquitto sin saber quien la consume, y RabbitMQ enruta los mensajes a colas según su contenido: todos los GPS a una cola, temperaturas mayores a 4 grados a otra y combustible menor al 20 por ciento a una tercera.

  Justificación técnica:

  Primero, el desacoplamiento permite que los vehículos sigan enviando datos aunque los consumidores no estén activos, porque los mensajes quedan almacenados en las colas.

  Segundo, el filtrado por topics y routing keys envía solo los mensajes relevantes a cada consumidor, reduciendo el procesamiento innecesario.

Tercero, las colas duraderas y el uso de QoS 1 en MQTT evitan pérdida de telemetría ante reinicios o caídas temporales de consumidores.

Conclusión:

Considero que la arquitectura pub/sub con brokers es la mas versátil para sistemas distribuidos modernos porque sirve para telemetría IoT, notificaciones en tiempo real, procesamiento asíncrono e integración de sistemas heterogéneos sin acoplamiento directo. Su límite es que no es ideal para consultas que requieran una respuesta inmediata y sincrónica, donde un modelo cliente-servidor o REST es mas adecuado.




```
sh-5.3$ python3 suscriptor_sqlite.py
Escuchando flota/+/gps, flota/+/temperatura y flota/+/combustible. Ctrl+C para detener
Suscriptor conectado a MQTT: Success
ALERTA TEMPERATURA: VH-002 = 7.0 C
ALERTA COMBUSTIBLE: VH-002 = 15.0 %
ALERTA TEMPERATURA: VH-002 = 7.0 C
ALERTA COMBUSTIBLE: VH-002 = 15.0 %
```

```
RabbitMQ: cola.alertas.temperatura <- {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.681591+00:00', 'temperature': 7.0, 'unit': 'C'}
RabbitMQ: cola.combustible.nivel <- {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.681811+00:00', 'fuel_level': 15.0, 'unit': '%'}
RabbitMQ: cola.gps.telemetria <- {'vehicle_id': 'VH-003', 'timestamp': '2026-10-30T13:27:41.682115+00:00', 'lat': -3.9879000000000002, 'lng': -79.2319, 'speed': 52}
RabbitMQ: cola.gps.telemetria <- {'vehicle_id': 'VH-001', 'timestamp': '2026-10-30T13:27:41.682766+00:00', 'lat': -4.0079, 'lng': -79.2119, 'speed': 42}
RabbitMQ: cola.gps.telemetria <- {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683201+00:00', 'lat': -3.9979000000000005, 'lng': -79.2219, 'speed': 47}
RabbitMQ: cola.alertas.temperatura <- {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683307+00:00', 'temperature': 7.0, 'unit': 'C'}
RabbitMQ: cola.combustible.nivel <- {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683397+00:00', 'fuel_level': 15.0, 'unit': '%'}
RabbitMQ: cola.gps.telemetria <- {'vehicle_id': 'VH-003', 'timestamp': '2026-10-30T13:27:41.683500+00:00', 'lat': -3.9879000000000002, 'lng': -79.2319, 'speed': 52}
```

```
Ronda 2: flota/VH-001/temperatura -> {'vehicle_id': 'VH-001', 'timestamp': '2026-10-30T13:27:41.68
t': 'C'}
Ronda 2: flota/VH-001/combustible -> {'vehicle_id': 'VH-001', 'timestamp': '2026-10-30T13:27:41.68
t': '%'}
Ronda 2: flota/VH-002/gps -> {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683201+00:0
': -79.2219, 'speed': 47}
Ronda 2: flota/VH-002/temperatura -> {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683307+00:00', 'temperature': 7.0, 'uni
t': 'C'}
Ronda 2: flota/VH-002/combustible -> {'vehicle_id': 'VH-002', 'timestamp': '2026-10-30T13:27:41.683397+00:00', 'fuel_level': 15.0, 'uni
t': '%'}
Ronda 2: flota/VH-003/gps -> {'vehicle_id': 'VH-003', 'timestamp': '2026-10-30T13:27:41.683500+00:00', 'lat': -3.9879000000000002, 'lng
': -79.2319, 'speed': 52}
Ronda 2: flota/VH-003/temperatura -> {'vehicle_id': 'VH-003', 'timestamp': '2026-10-30T13:27:41.683593+00:00', 'temperature': 3.0, 'uni
t': 'C'}
Ronda 2: flota/VH-003/combustible -> {'vehicle_id': 'VH-003', 'timestamp': '2026-10-30T13:27:41.683716+00:00', 'fuel_level': 50.0, 'uni
t': '%'}
sh-5.3$
```

Process ID (PID): 2961
Command line: /bin/sh
Shell integration: No
Show Environment Contributions Show Details

```
sh-5.3$ python3 verificar_resultados.py
gps_data: 7 registros
temp_data: 7 registros
fuel_data: 7 registros
Alertas de temperatura: [('VH-001', 7.0), ('VH-002', 7.0), ('VH-002', 7.0)]
Alertas de combustible: [('VH-002', 15.0), ('VH-002', 15.0), ('VH-002', 15.0)]
sh-5.3$
```

Overview | Connections | Channels | Exchanges | **Queues and Streams** | Admin

Queues

All queues (3)

Pagination

Page 1 of 1 - Filter: Regex ?

Overview					Messages			Message rates			+/-
Virtual host	**Name**	**Type**	Features	**State**	**Ready**	**Unacked**	**Total**	**incoming**	**deliver / get**	**ack**	
/	**cola.alertas.temperatura**	classic	D	running	2	0	2	0.00/s			
/	**cola.combustible.nivel**	classic	D	running	2	0	2	0.00/s			
/	**cola.gps.telemetria**	classic	D	running	6	0	6	0.00/s			

Add a new queue

HTTP API | Documentation | Tutorials | New releases | Commercial edition | Commercial support | Discussions | Discord | Plugins | GitHub

6. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC